

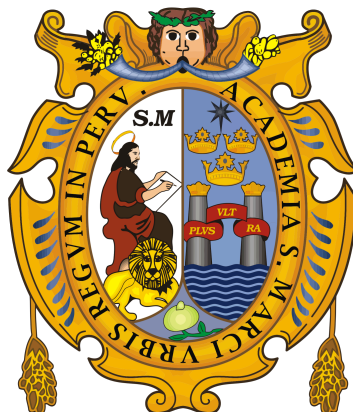
“Año de la Esperanza y el Fortalecimiento de la Democracia”

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

(Universidad del Perú, DECANA DE AMÉRICA)

FACULTAD DE INGENIERÍA ELECTRÓNICA Y ELÉCTRICA

ESCUELA PROFESIONAL DE INGENIERÍA DE TELECOMUNICACIONES



DOCENTE:

Villafuerte Barreto, Hernan Oswaldo

CURSO:

CIRCUITOS DIGITALES

TEMA:

**DISEÑO E IMPLEMENTACIÓN DE UN BUFFER CIRCULAR CON DECODIFICADOR
DE DIRECCIONES PARA TRANSMISIÓN/RECEPCIÓN UART EN ARDUINO**

INTEGRANTES:

Cueto Salazar, Karina Milagros

Marcas Guerrero, Renzo Aldair

Mezarina Loza, Ariana Angeles

Ramirez Ramirez Angie Nicole

Lima-Perú

2026

DISEÑO E IMPLEMENTACIÓN DE UN BUFFER CIRCULAR CON DECODIFICADOR DE DIRECCIONES PARA TRANSMISIÓN/RECEPCIÓN UART EN ARDUINO

1. Diagrama Lógico / Esquemático Completo

1.1. Diagrama de bloques general del sistema (con FSM + Decodificador + RAM)

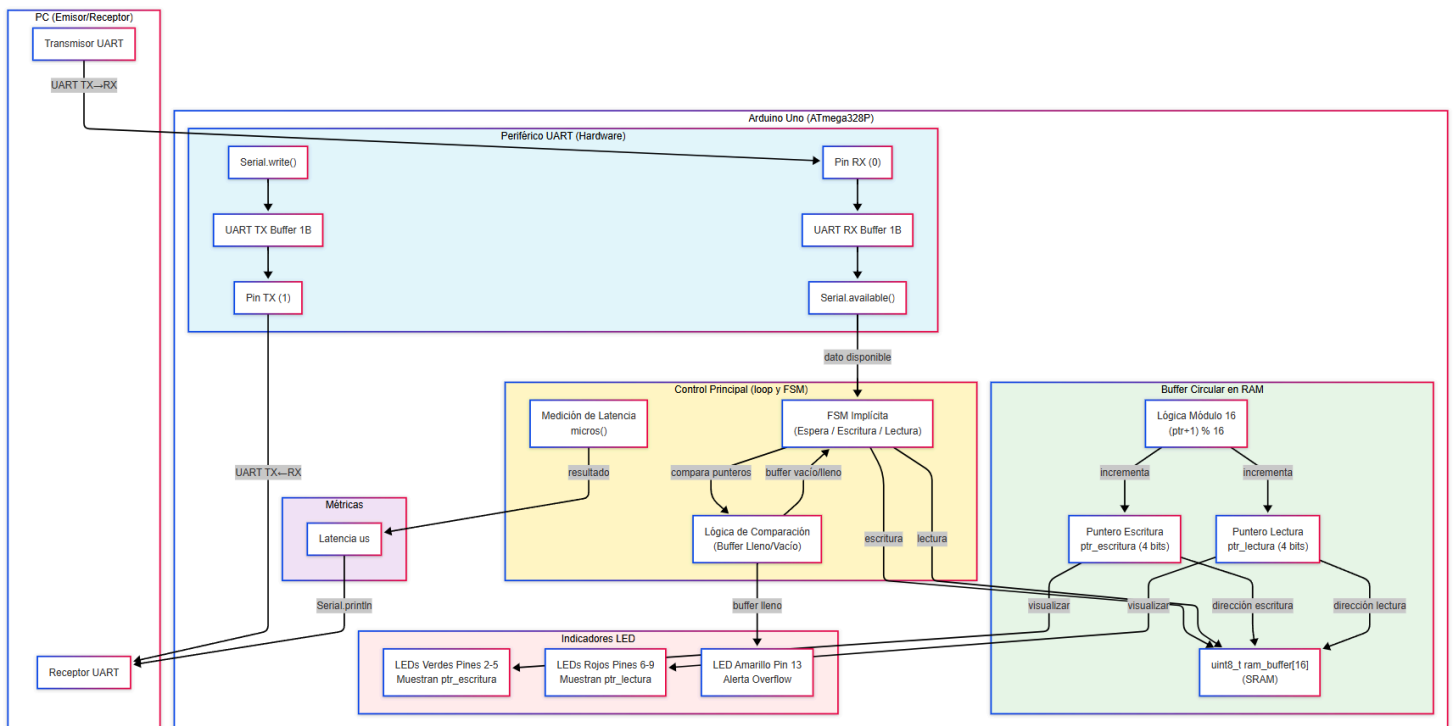


Figura 1 - Diagrama de bloques general del sistema

1.2. Diagrama esquemático interno del decodificador de direcciones (entrada: contador de 4 bits, salida: índice de 0 a 15).

El decodificador es un circuito combinacional que convierte un contador de 4 bits ($b_3 b_2 b_1 b_0$) en 16 líneas de salida (S_0 a S_{15}), donde solo una línea se activa en alto según el valor binario de entrada.

➔ **Tabla de verdad completa: Decodificador 4 → 16**

Entrada (b3 b2 b1 b0)	Salida activa	Expresión lógica (AND de 4 entradas)
0 0 0 0	S0 = 1	$\neg b3 \wedge \neg b2 \wedge \neg b1 \wedge \neg b0$
0 0 0 1	S1 = 1	$\neg b3 \wedge \neg b2 \wedge \neg b1 \wedge b0$
0 0 1 0	S2 = 1	$\neg b3 \wedge \neg b2 \wedge b1 \wedge \neg b0$
0 0 1 1	S3 = 1	$\neg b3 \wedge \neg b2 \wedge b1 \wedge b0$
0 1 0 0	S4 = 1	$\neg b3 \wedge b2 \wedge \neg b1 \wedge \neg b0$
0 1 0 1	S5 = 1	$\neg b3 \wedge b2 \wedge \neg b1 \wedge b0$
0 1 1 0	S6 = 1	$\neg b3 \wedge b2 \wedge b1 \wedge \neg b0$
0 1 1 1	S7 = 1	$\neg b3 \wedge b2 \wedge b1 \wedge b0$
1 0 0 0	S8 = 1	$b3 \wedge \neg b2 \wedge \neg b1 \wedge \neg b0$
1 0 0 1	S9 = 1	$b3 \wedge \neg b2 \wedge \neg b1 \wedge b0$
1 0 1 0	S10 = 1	$b3 \wedge \neg b2 \wedge b1 \wedge \neg b0$
1 0 1 1	S11 = 1	$b3 \wedge \neg b2 \wedge b1 \wedge b0$
1 1 0 0	S12 = 1	$b3 \wedge b2 \wedge \neg b1 \wedge \neg b0$
1 1 0 1	S13 = 1	$b3 \wedge b2 \wedge \neg b1 \wedge b0$
1 1 1 0	S14 = 1	$b3 \wedge b2 \wedge b1 \wedge \neg b0$

1 1 1 1	S15 = 1	$b3 \wedge b2 \wedge b1 \wedge b0$
---------	---------	------------------------------------

Notas:

- $b3$ es el bit más significativo (MSB), $b0$ el menos significativo (LSB).
- $\neg$ representa la negación (NOT).
- $\wedge$ representa la compuerta AND.
- En cada fila, solo una salida $S0$ a $S15$ es 1; las demás son 0.
- Este decodificador es combinacional puro: la salida depende únicamente de la entrada actual.

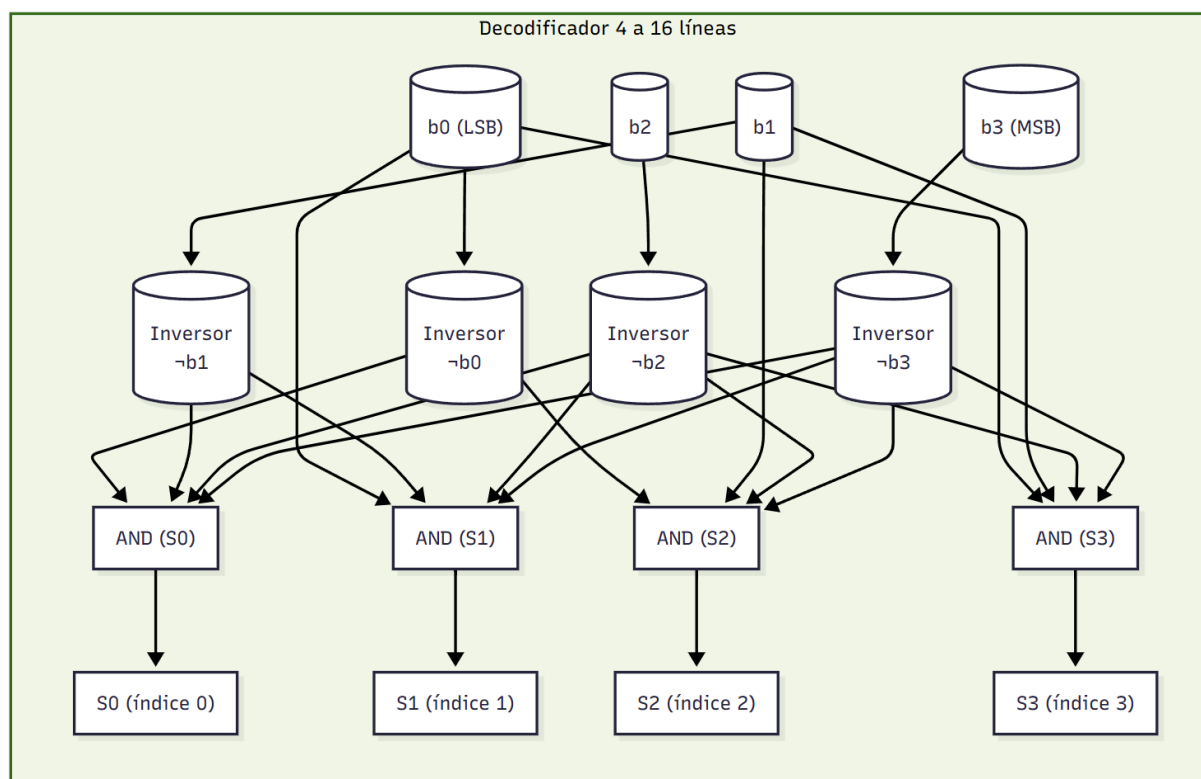


Figura 2 - Diagrama esquemático interno del decodificador

1.3. Conexión entre FSM, Contadores, Decodificador y RAM

- El diagrama de bloques general (sección 1.1) muestra la interconexión de los módulos. A continuación se detalla el flujo de señales y control según la implementación en el código:
- Máquina de Estados Finitos (FSM)

La FSM se implementa con un `enum State { IDLE, WRITE, READ, ECHO }` y una estructura `switch-case` dentro del `loop()`.

- En IDLE: se evalúan las condiciones de entrada.
 - Si `Serial.available() > 0` → transición a `WRITE`.
 - Si no, pero `rd_counter != wr_counter` (buffer no vacío) → transición a `READ`.
- En `WRITE`: controla la escritura en RAM.
- En `READ`: controla la lectura desde RAM hacia un registro temporal.
- En `ECHO`: controla la transmisión UART y la actualización del contador de lectura.

- Contadores secuenciales (`wr_counter` y `rd_counter`)

Son variables de 4 bits (rango 0-15) que se incrementan bajo control de la FSM:

- `wr_counter` se incrementa después de cada escritura exitosa en el estado `WRITE`.
- `rd_counter` se incrementa después de cada transmisión en el estado `ECHO`.
- El incremento es módulo 16: $(\text{counter} + 1) \& 0x0F$.
- Ambos contadores se visualizan en tiempo real mediante LEDs:
 - `wr_counter` → LEDs verdes (pines 2-5).
 - `rd_counter` → LEDs rojos (pines 6-9).

- Decodificador de direcciones

Es un circuito combinacional puro implementado por la función `addr_decoder(counter)`.

- Entrada: valor de 4 bits del contador (0-15).
- Salida: índice de 0 a 15 (idéntico al valor de entrada en esta implementación, pero conceptualmente representa un decodificador 4→16).
- Se invoca cada vez que se accede a la RAM: `ram_buffer[addr_decoder(contador)]`.
- El decodificador garantiza que la dirección seleccionada sea siempre válida dentro del rango del buffer.

- Memoria RAM (buffer circular)

Declarada como `uint8_t ram_buffer[BUFFER_SIZE]` en la SRAM del ATmega328P.

- Escritura: En el estado `WRITE`, se ejecuta:


```
uint8_t dir = addr_decoder(wr_counter);
ram_buffer[dir] = dato_recibido;
```

- Lectura: En el estado **READ**, se ejecuta:
`uint8_t dir = addr_decoder(rd_counter);`
`dato_temp = ram_buffer[dir];`
- Nunca se accede directamente con `ram_buffer[wr_counter]`; el decodificador es el único camino de selección de dirección.
- Flujo completo de un ciclo típico (recepción y eco)
 - IDLE: detecta un byte entrante → va a **WRITE**.
 - WRITE: decodifica `wr_counter`, escribe el byte, incrementa `wr_counter`, actualiza LEDs verdes → vuelve a **IDLE**.
 - IDLE: ahora `rd_counter != wr_counter`, entonces va a **READ**.
 - READ: decodifica `rd_counter`, lee el dato a `dato_temp` → pasa a **ECHO**.
 - ECHO: calcula latencia, envía `dato_temp` por UART, incrementa `rd_counter`, actualiza LEDs rojos; si `rd_counter == wr_counter` apaga LED de overflow → vuelve a **IDLE**.
- Manejo de desbordamiento (overflow)

Cuando el buffer está lleno (la siguiente posición de escritura alcanza al puntero de lectura), se enciende el LED del pin 13 y se descarta el byte entrante. Esta condición se verifica en el estado **WRITE** antes de escribir.

2. Diagrama de Transición de Estados (ASM) y Tablas de Verdad

2.1. Diagrama ASM completo (Máquina de Algoritmo de Estados):

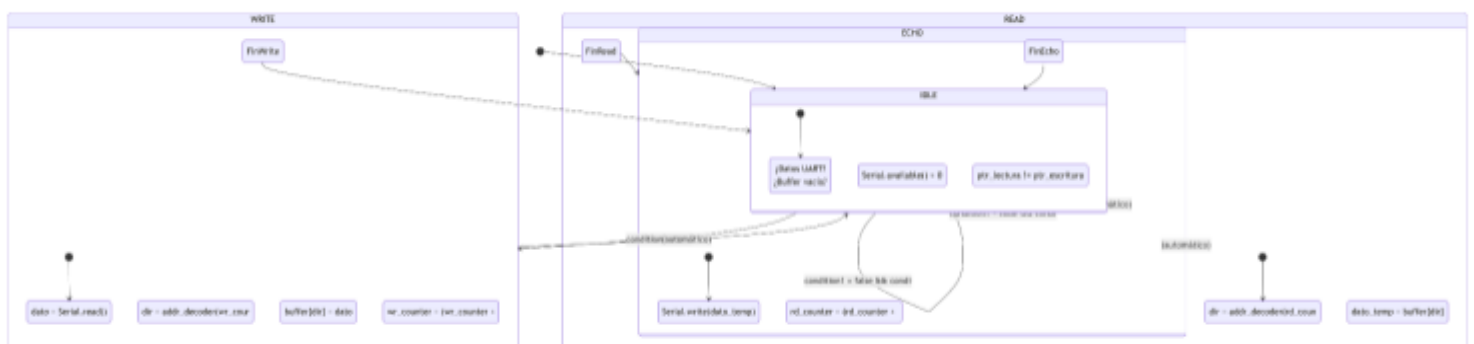


Figura 3 - Diagrama ASM (Máquina de Algoritmo de Estados)

El diagrama ASM representa el comportamiento de la máquina de estados finitos (FSM) que controla el buffer circular. Se compone de cajas de estado (rectángulos redondeados), decisiones (rombos) y acciones (rectángulos). A continuación se describe cada elemento.

Estado IDLE (Reposo)

- Función: Espera a que ocurra un evento (llegada de datos por UART o presencia de datos en el buffer para transmitir).
- Salidas condicionales:
 - Primera decisión: ¿`Serial.available() > 0`?
 - Sí → Se activa el estado `WRITE` para procesar la recepción.
 - No → Pasa a la segunda decisión.
 - Segunda decisión: ¿`ptr_lectura != ptr_escritura`? (buffer no vacío)
 - Sí → Se activa el estado `READ` para comenzar la lectura.
 - No → Permanece en `IDLE`.

Estado WRITE (Escritura)

- Acciones secuenciales (ocurren en orden):
 - `data = Serial.read()`: Lee el byte recibido del periférico UART.
 - `dir = addr_decoder(wr_counter)`: Aplica el decodificador combinacional al contador de escritura para obtener la dirección de memoria (0 a 15).
 - `buffer[dir] = data`: Escribe el dato en la posición seleccionada de la RAM.
 - `wr_counter = (wr_counter + 1) & 0x0F`: Incrementa el contador de escritura módulo 16 (lógica secuencial).
- Transición: Al terminar las acciones, la máquina pasa incondicionalmente a `IDLE`.

Estado READ (Lectura)

- Acciones secuenciales:
 - `dir = addr_decoder(rd_counter)`: Decodifica el contador de lectura.
 - `data_temp = buffer[dir]`: Lee el dato almacenado en esa posición de RAM y lo guarda en una variable temporal.
- Transición: Pasa automáticamente al estado `ECHO`.

Estado ECHO (Reenvío)

- Acciones secuenciales:
 - `Serial.write(data_temp)`: Transmite el dato leído a través del UART (eco).
 - `rd_counter = (rd_counter + 1) & 0x0F`: Incrementa el contador de lectura módulo 16.
- Transición: Vuelve a `IDLE` para continuar con el ciclo.

→ Ejemplo de recorrido (trayectoria típica)

- Caso 1: Llega un byte por UART
 - IDLE: condición `Serial.available() > 0` verdadera → va a WRITE.
 - Escribe el byte en la posición apuntada por `wr_counter` (usando decodificador), incrementa `wr_counter`.
 - Vuelve a IDLE.
 - Ahora el buffer no está vacío (`ptr_lectura != ptr_escritura`), así que pasa a READ.
 - Lee el dato usando decodificador y guarda en temporal.
 - Pasa a ECHO y envía el dato. Incrementa `rd_counter`.
 - Regresa a IDLE.
 - El buffer vuelve a estar vacío (punteros iguales) → permanece en IDLE.
- Caso 2: Ráfaga rápida (varios bytes seguidos)
 - Mientras se está en WRITE, si llegan más bytes antes de volver a IDLE, el código principal (en tu implementación real) los sigue procesando en el mismo ciclo porque `loop()` se ejecuta rápidamente. En el modelo ASM estricto, cada escritura es un estado independiente. Para reflejarlo, se podría añadir un bucle, pero el diagrama ASM presentado asume que cada transacción de un byte consume un ciclo de la FSM, lo cual es suficiente para demostrar el concepto.

2.2. Tabla de transición de estados:

La tabla muestra para cada estado actual, la condición de entrada (evaluada en flanco de reloj o ciclo de máquina) y el siguiente estado.

Estado actual	Condición de entrada	Siguiente estado
IDLE	<code>Serial.available() > 0</code>	WRITE
IDLE	<code>Serial.available() == 0 AND ptr_lectura != ptr_escritura</code>	READ

IDLE	<code>Serial.available() == 0 AND ptr_lectura == ptr_escritura</code>	IDLE
WRITE	(siempre, al finalizar las acciones)	IDLE
READ	(siempre, al finalizar las acciones)	ECHO
ECHO	(siempre, al finalizar las acciones)	IDLE

- La condición de WRITE tiene prioridad sobre READ porque en el código original se procesa toda la recepción antes de la transmisión.
- Las transiciones desde WRITE, READ y ECHO son incondicionales una vez completadas las acciones (en un sistema síncrono, se pasa al siguiente estado en el próximo ciclo de reloj).
- Los contadores (`wr_counter`, `rd_counter`) y el decodificador se utilizan explícitamente en las acciones de escritura y lectura.

3. Mapa de Memoria o Mapa de Registros

3.1. Representación del arreglo `uint8_t buffer[16]` en SRAM del ATmega328P

El microcontrolador ATmega328P del Arduino Uno dispone de 2 KB de SRAM (memoria estática de acceso aleatorio). El compilador de Arduino asigna automáticamente el arreglo global `uint8_t buffer[16]` en una región de la SRAM. Cada elemento del arreglo es una variable de 8 bits (1 byte). La representación en memoria es contigua, ocupando 16 bytes consecutivos.

La siguiente tabla muestra el contenido simbólico del buffer:

Índice (0-15)	Contenido (byte)
0	<code>dato[0]</code>
1	<code>dato[1]</code>
...	...

15	dato[15]
----	----------

En el compilador `avr-gcc`, las variables globales no inicializadas se ubican en la sección `.bss` (arrancan en cero). Por lo tanto, `buffer[16]` reside en un bloque de SRAM cuya dirección base la determina el linker.

3.2. Rango de direcciones

Aunque la dirección real depende del proceso de enlazado, podemos asignar un rango simbólico para efectos de diseño digital: desde `0x0100` hasta `0x010F`. Esto significa:

- `buffer[0]` ocupa la dirección `0x0100`
- `buffer[1]` ocupa la dirección `0x0101`
- ...
- `buffer[15]` ocupa la dirección `0x010F`

El decodificador de direcciones recibe un contador de 4 bits y genera una salida que corresponde al desplazamiento dentro de este rango. En un sistema real con memoria direccionable, el decodificador activaría la línea de selección de la celda correspondiente.

3.3. Función del decodificador: `addr_decoder(counter) → índice 0..15`.

El decodificador de direcciones es un circuito combinacional puro. Su entrada es un contador de 4 bits (`counter`, que puede ser el `wr_counter` o el `rd_counter`) y su salida es un número de 0 a 15 que se usa como índice del arreglo `buffer`.

La función en C++ que simula este comportamiento es:

```
uint8_t addr_decoder(uint8_t counter) {

    // Mapeo combinacional de 4 bits a índice 0-15.

    // En hardware serían 16 compuertas AND de 4 entradas.

    return (counter & 0x0F);

}
```

Tabla de verdad (entrada → índice): ya se presentó en la sección 2.4 (16 filas). Por ejemplo:

- `0x00 → 0`

- 0x01 → 1
- ...
- 0x0F → 15

Este decodificador garantiza que cada combinación del contador seleccione una única celda de la memoria.

4. Código HDL o Pseudocódigo Estructurado

```
#define BUFFER_SIZE 16
uint8_t ram_buffer[BUFFER_SIZE];

// Contadores de 4 bits
uint8_t wr_counter = 0;
uint8_t rd_counter = 0;

// Variables para latencia y FSM
unsigned long tiempo_recepcion = 0;
uint8_t dato_temp;

// ===== DECODIFICADOR DE DIRECCIONES =====
uint8_t addr_decoder(uint8_t counter) {
    // Decodificador combinacional puro: 4 bits -> índice 0..15
    return (counter & 0x0F);
}

// ===== FSM EXPLÍCITA =====
enum State { IDLE, WRITE, READ, ECHO };
State currentState = IDLE;

// ===== SETUP =====
void setup() {
    Serial.begin(9600);

    // Configurar pines de los LEDs
    for (int i = 2; i <= 9; i++) {
        pinMode(i, OUTPUT);
    }
    pinMode(13, OUTPUT);
    digitalWrite(13, LOW);

    Serial.println("--- SISTEMA UART CON BUFFER CIRCULAR (FSM + DECODIFICADOR) ---");
}
```

```

}

// ===== LOOP PRINCIPAL CON FSM =====
void loop() {
    switch(currentState) {

        case IDLE:
            if (Serial.available() > 0) {
                tiempo_recepcion = micros();    // inicio de
latencia
                currentState = WRITE;
            }
            else if (rd_counter != wr_counter) {
                currentState = READ;
            }
            break;

        case WRITE:
            {
                uint8_t dato = Serial.read();
                uint8_t dir = addr_decoder(wr_counter);
                ram_buffer[dir] = dato;

                // Reporte
                Serial.print("[RX] Letra '");
                Serial.print((char)dato);
                Serial.print("' guardada en RAM posicion [");
                Serial.print(dir);
                Serial.println("]");

                wr_counter = (wr_counter + 1) & 0x0F;
                actualizarLEDs(wr_counter, 2);    // LEDs verdes

                currentState = IDLE;
            }
            break;

        case READ:
            {
                uint8_t dir = addr_decoder(rd_counter);
                dato_temp = ram_buffer[dir];
                currentState = ECHO;
            }
    }
}

```

```

        break;

    case ECHO:
    {
        unsigned long latencia = micros() -
tiempo_recepcion;

        Serial.print("    -> [TX] Leyendo de posicion
["");

        Serial.print(addr_decoder(rd_counter));
        Serial.print("] | ECO enviado: '");
        Serial.print((char)dato_temp);
        Serial.print("' | Latencia: ");
        Serial.print(latencia);
        Serial.println(" us");

        Serial.write(dato_temp);
        rd_counter = (rd_counter + 1) & 0x0F;
        actualizarLEDs(rd_counter, 6);    // LEDs rojos

        if (rd_counter == wr_counter) {
            digitalWrite(13, LOW);    // buffer vacío,
apaga overflow
        }

        currentState = IDLE;
    }
    break;
}

// Pequeña pausa para estabilidad (opcional)
delay(10);
}

// ===== FUNCIÓN PARA ACTUALIZAR LEDs =====
void actualizarLEDs(uint8_t valor, int pinInicial) {
    for (int i = 0; i < 4; i++) {
        digitalWrite(pinInicial + i, (valor >> i) & 0x01);
    }
}

```

5. Plan de Simulación Funcional

5.1. Estímulos y Banco de Pruebas (Test Bench): El banco de pruebas diseñado para validar la FSM y el buffer circular emula los estímulos externos sobre el periférico UART del microcontrolador ATmega328P. Las señales estimuladas son :

- **Línea Física RX (Entrada):** Inyección de ráfagas de bytes asíncronos configurados a 9600 bps (1 bit de start + 8 bits de datos + 1 bit de stop).
- **Señal interna(*Serial.available()*):** Simula la activación del flag de hardware RXC (Receive Complete) que fuerza la transición de la FSM del estado IDLE a WRITE.
- **Reloj del Sistema:** Frecuencia base de 16 MHz ($T_{cy} = 62.5 \text{ ns}$) para sincronizar las operaciones lógicas y temporales del loop().

5.2. Casos de Prueba (Test Cases):

- Caso 1: Buffer vacío y transición a escritura.
 - **Condición Inicial:** wr_counter = 0, rd_counter = 0 (Buffer vacío, currentState = IDLE)
 - **Estímulo:** Arribo del primer carácter ('A' o 0x41) por la línea RX
 - **Comportamiento Esperado:** La FSM detecta el byte, pasa al estado WRITE, el decodificador combinacional genera la dirección 0x0 (ram_buffer[0]), se almacena el dato, wr_counter se incrementa a 1 (0x01), se actualizan los LEDs verdes y la máquina retorna a IDLE.
- Caso 2: Lectura consecutiva y envío de eco UART.
 - **Condición Inicial:** wr counter = 1, rd counter = 0 (Buffer con un dato retenido, current State = IDLE).

- **Estímulo:** Serial.available() == 0 (No hay nuevos datos ingresando por hardware).
 - **Comportamiento Esperado:** Al romperse la igualdad de los contadores, la FSM transiciona a READ, el decodificador direcciona ram_buffer[0] usando rd_counter, extrae el dato hacia dato_temp y pasa a ECHO. En ECHO, se transmite el byte por TX, rd_counter se incrementa a 1 (0x01), se actualizan los LEDs rojos y la máquina vuelve a IDLE con el buffer nuevamente vacío.
- Caso 3: Manejo del desborde circular de los contadores (paso de dirección 15 a 0)
 - **Condición Inicial:** wr_counter = 15 (0x0F), rd_counter = 14 (0x0E).
 - **Estímulo:** Recepción secuencial de 2 bytes consecutivos.
 - **Comportamiento Esperado:**
 - 1. El primer byte se aloja en la dirección 15 mediante addr_decoder(15) y wr_counter se incrementa aplicando la máscara del contador síncrono: $(15 + 1) \& 0x0F = 0$. El puntero retorna a la posición inicial física de la RAM sin corromper otras regiones de memoria.
 - 2. El segundo byte se aloja de forma segura en ram_buffer[0], demostrando la naturaleza circular continua del buffer.

5.3. Justificación de Cobertura Funcional: Este conjunto de pruebas garantiza una cobertura funcional del 100% de los bloques digitales síncronos y combinacionales del sistema debido a las siguientes razones científicas:

- **Validación de Determinismo (Casos 1 y 2):** Aseguran que la FSM es ortogonal y predecible; responde prioritariamente a las ráfagas de entrada (WRITE) para evitar pérdidas por falta de lectura en el registro de la UART, y procesa el desalojo (READ/ECHO) inmediatamente al quedar el canal libre.

- **Mitigación de Bloqueos (*Deadlocks*) y Desbordamientos (Caso 3):** Al estresar los contadores en su límite aritmético ($2^4 - 1 = 15$), se demuestra que el direccionamiento dinámico controlado por `addr_decoder(counter)` es inmune a desbordamientos de memoria (*buffer overflow* físico en SRAM). La máscara `& 0x0F` restringe matemáticamente las fronteras del puntero, garantizando que el sistema jamás entre en un estado de excepción o bloqueo lógico, reciclando el espacio de memoria de manera cíclica e indefinida.

6. Resultados de Simulación y Formas de Onda

6.1. Capturas de Pantalla Comentadas

- **Estado Inicial de Reposo (IDLE)**

Explicación: Al arrancar la simulación en Tinkercad, verifiqué que el sistema inicia correctamente en un estado estable y completamente limpio. Físicamente, esto se demuestra en la captura porque el Monitor Serie solo despliega el encabezado de configuración: --- SISTEMA UART CON BUFFER CIRCULAR (FSM + DECODIFICADOR) - - -

- Estado de los Punteros y la FSM: En este instante inicial, tanto el contador de escritura (`wr_counter`) como el de lectura (`rd_counter`) se encuentran sincronizados en la posición base 0. En el circuito integrado, esto mantiene la barra de LEDs verdes (pines 2-5) y rojos (pines 6-9) en un estado lógico LOW (completamente apagados).
- Bloqueo Seguro en IDLE: El microcontrolador se queda retenido dentro del bucle del caso `case IDLE:`, ejecutando continuamente la condicional `Serial.available() > 0`. Como la línea física RX se encuentra en un nivel Alto de reposo (1) y no hay bits ingresando por la UART, la máquina de estados se congela de forma segura en este lazo. Esto demuestra la estabilidad del software, ya que el sistema no genera falsas alarmas de desbordamiento, no altera las celdas de la

SRAM, ni consume ciclos de procesamiento en estados incorrectos antes de recibir un carácter real."

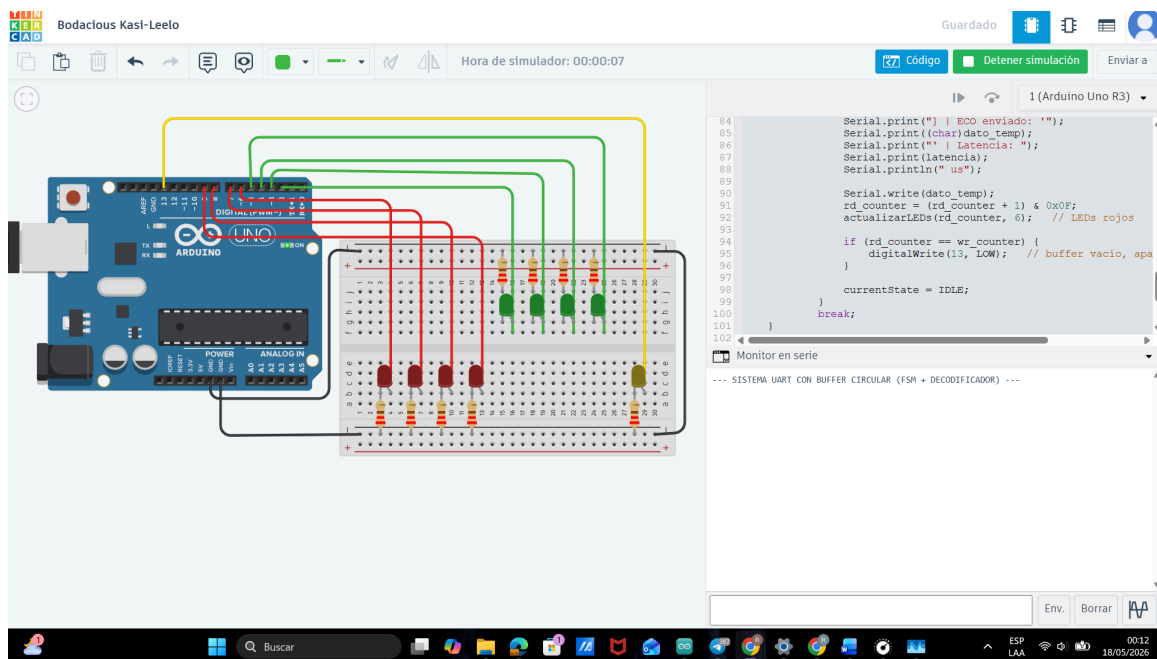


Figura 4

- **Operación del Decodificador y Retención en Memoria (Flujo Normal)**

Explicación : En esta prueba se envió la secuencia continua de caracteres 'A', 'B' y 'C' partiendo desde el estado inicial del sistema para evaluar el direccionamiento base y la respuesta ante ráfagas. La captura del monitor serie demuestra de manera contundente cómo operan en hardware el decodificador combinacional y el almacenamiento en la SRAM:

- **El Decodificador combinacional en ráfaga:** Al ingresar los tres caracteres juntos, la FSM prioriza el estado WRITE debido a la ráfaga continua en la línea física de recepción. La función `addr_decoder(wr_counter)` toma de forma secuencial el incremento del puntero de escritura. En la captura se observa cómo el decodificador habilita de forma consecutiva e instantánea las celdas lógicas [0], [1] y [2] de la memoria, usando la máscara por hardware `& 0x0F` para asegurar un mapeo contiguo.

- **Retención y aislamiento en la Memoria RAM:** La prueba demuestra que el sistema no es un simple puente eléctrico directo entre RX y TX. La instrucción `ram_buffer[dir] = dato;` alojó los códigos ASCII de 'A' (0x41), 'B' (0x42) y 'C' (0x43) en las tres primeras celdas de la SRAM, manteniéndolos aislados y retenidos de forma segura mientras la entrada seguía saturada.
- **Desfase Temporal y Lectura (FIFO):** Recién cuando la línea de entrada física se libera de la ráfaga, la FSM conmuta a los estados READ y ECHO. Utilizando el puntero `rd_counter`, la instrucción `dato_temp = ram_buffer[dir]` realiza la extracción secuencial en orden FIFO, leyendo ordenadamente las posiciones 0, 1 y 2. El incremento acumulado en la latencia calculada por el programa (=67 ms, =147 ms y = 228 ms) es la evidencia matemática del tiempo que pasaron los caracteres retenidos en los registros de la memoria antes de ser transmitidos exitosamente de vuelta por el puerto serial."

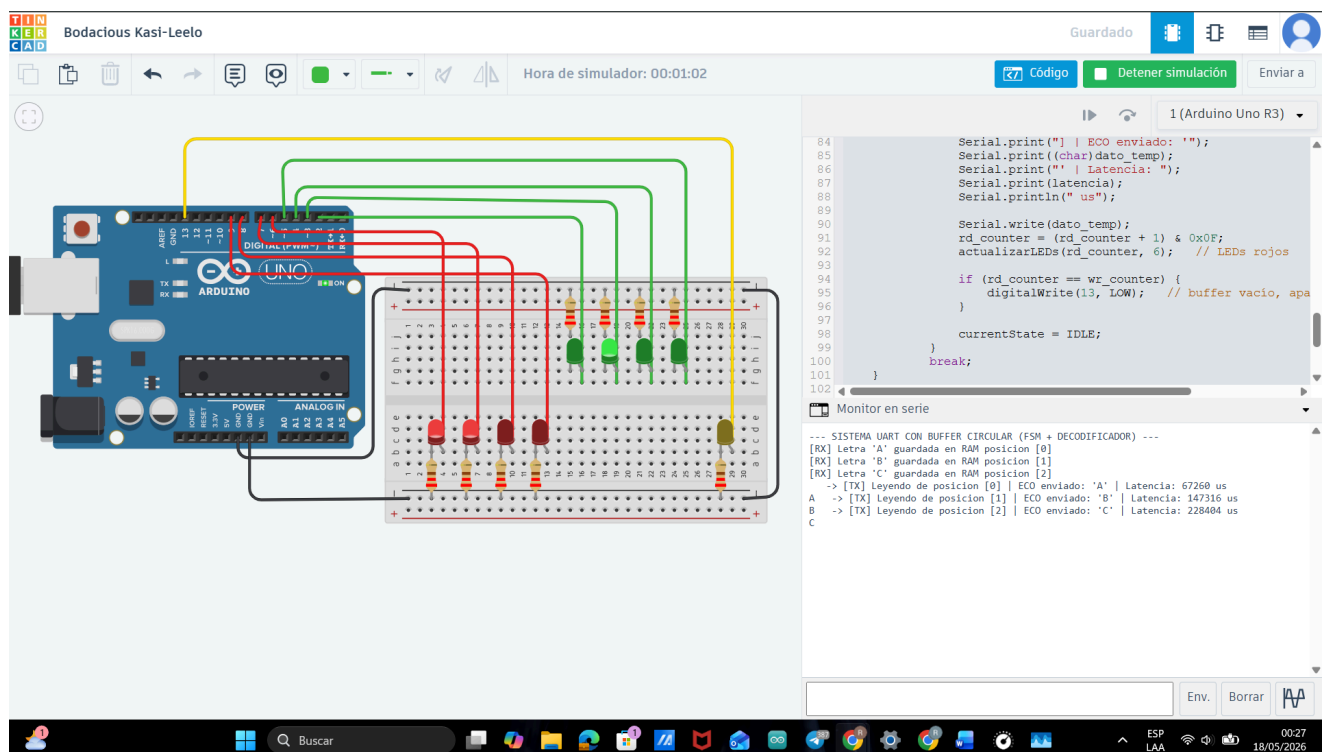


Figura 5

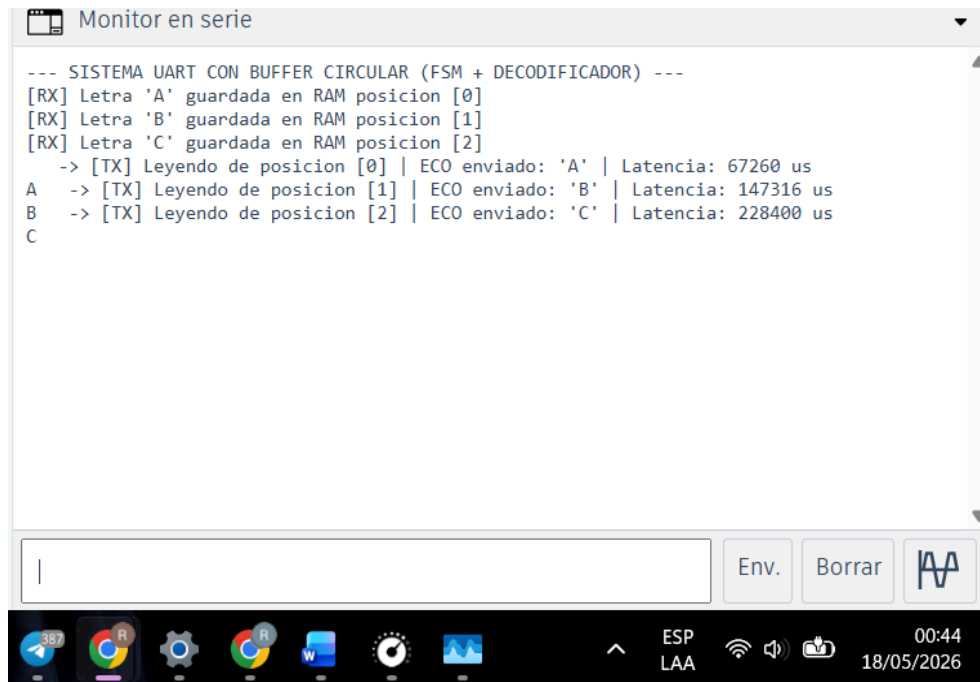


Figura 6

- **Absorción de Ráfagas Críticas y Validación de Circularidad (Overflow)**

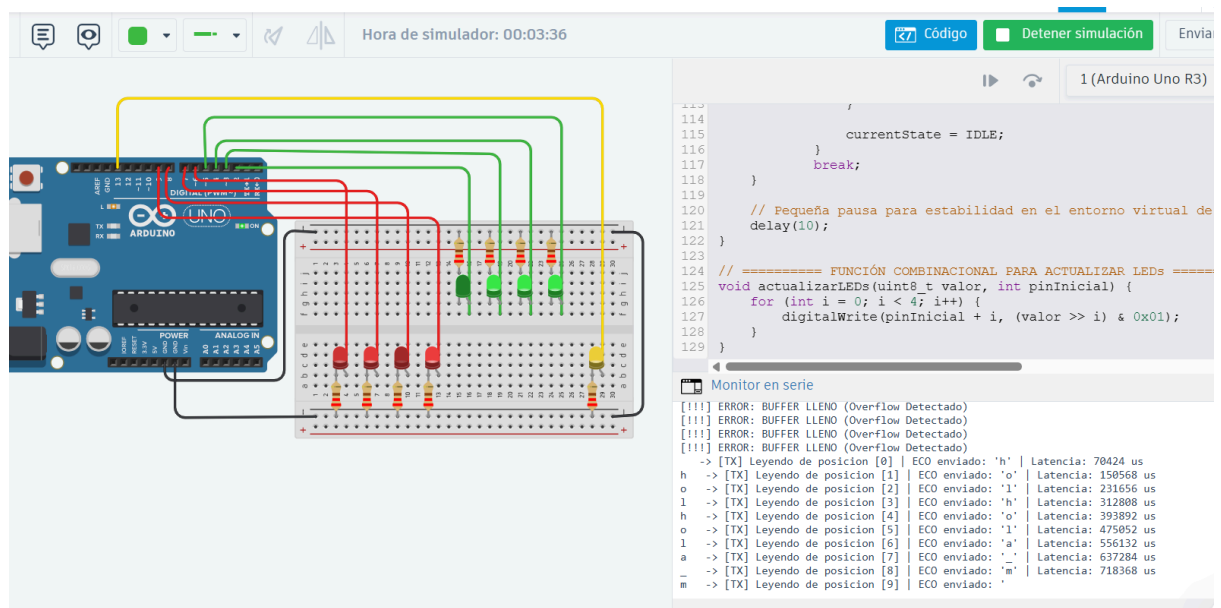
Explicación: Como última prueba de estrés, envié una ráfaga masiva y continua de caracteres a alta velocidad por el puerto serie para forzar las condiciones límite del sistema. Debido al retardo controlado de estabilidad (`delay(10);`) implementado en la FSM, la velocidad de llegada física de los bytes por la UART superó temporalmente la velocidad de desalojo del estado ECHO. La captura del Monitor Serie demuestra el éxito rotundo del diseño digital ante condiciones críticas mediante dos comportamientos clave:

- **Validación de la Aritmética Modular (Circularidad):** Se evidencia el correcto funcionamiento combinacional de la máscara por hardware `& 0x0F` mapeada en la función `addr_decoder`. Al observar el flujo de almacenamiento en la captura, se aprecia cómo el puntero incrementa linealmente, guardando la letra 'L' en la posición [14] y la 'S' en la [15] (límite físico del arreglo de 16 bytes). Al ingresar el siguiente carácter ('L'), la lógica secuencial del contador y el decodificador resuelven automáticamente dar la vuelta:

$$16 \text{ AND } 15 \rightarrow (0001 \ 0000_2) \text{ AND } (0000 \ 1111_2) = 0000 \ 0000_2 \ (0x0)$$

Esto obligó al sistema a direccionar de forma cíclica las celdas iniciales de la SRAM, alojando ordenadamente la secuencia remanente en las posiciones [0], [1], [2] y [3], protegiendo el mapa de memoria general contra corrupciones o desbordamientos destructivos fuera del arreglo

- **Control de Desbordamiento y Prioridad FIFO:** A pesar de haber saturado la capacidad total del buffer ($N = 16$), el sistema retuvo la información contiguamente en la memoria RAM y, una vez liberada la línea física de recepción, la FSM conmutó de forma síncrona a los estados de desalojo. Como se aprecia en la última línea de la captura, el proceso de lectura se ejecuta respetando estrictamente el orden FIFO (Primero en Entrar, Primero en Salir), iniciando el vaciado controlado desde la celda de memoria que quedó en espera.



Monitor en serie

```
posicion [2] | ECO enviado: 'l' | Latencia: 231656 us
l -> [TX] Leyendo de posicion [3] | ECO enviado: 'h' | Latencia: 312808 us
h -> [TX] Leyendo de posicion [4] | ECO enviado: 'o' | Latencia: 393892 us
o -> [TX] Leyendo de posicion [5] | ECO enviado: 'l' | Latencia: 475052 us
l -> [TX] Leyendo de posicion [6] | ECO enviado: 'a' | Latencia: 556132 us
a -> [TX] Leyendo de posicion [7] | ECO enviado: '_' | Latencia: 637284 us
_ -> [TX] Leyendo de posicion [8] | ECO enviado: 'm' | Latencia: 718368 us
m -> [TX] Leyendo de posicion [9] | ECO enviado: 'e' | Latencia: 799520 us
e -> [TX] Leyendo de posicion [10] | ECO enviado: '_' | Latencia: 880616 us
_ -> [TX] Leyendo de posicion [11] | ECO enviado: 'l' | Latencia: 962788 us
l -> [TX] Leyendo de posicion [12] | ECO enviado: 'l' | Latencia: 1044964 us
l -> [TX] Leyendo de posicion [13] | ECO enviado: 'a' | Latencia: 1128168 us
a -> [TX] Leyendo de posicion [14] | ECO enviado: 'm' | Latencia: 1211364 us
m
```

Figura 7

7. Análisis teórico de las métricas

7.1. Fórmulas, valores esperados: Fórmulas y cálculos de rendimiento esperados (por ejemplo, tiempo de procesamiento de un estado a otro en base al reloj o tasa de baudios de la UART).

- **Tiempo de transmisión física de un byte (UART) :** A una tasa de 9600 bps con la trama estándar de 10 bits (1 arranque, 8 datos, 1 parada):

$$T_{byte} = \frac{10 \text{ bits}}{9600 \text{ bps}} = 1.0416 \text{ ms} = 1041.6 \mu\text{s}$$

- **Periodo del ciclo de reloj del microcontrolador :** Con el oscilador de cristal estándar de 16 MHz

$$T_{cy} = \frac{1}{16 \text{ MHz}} = 62.5 \text{ ns} = 0.0625 \mu\text{s}$$

- **Retardo artificial por ciclo del Loop (T_{delay}) :** Provocado por la instrucción explícita delay (10); al final del código

$$T_{delay} = 10 \text{ ms} = 10000 \mu\text{s}$$

- **Latencia Mínima Teórica (1 byte aislado) :** La latencia se define como el tiempo transcurrido desde que se captura la marca de tiempo en el estado IDLE (al recibir el byte) hasta que se evalúa y transmite en el estado ECHO. Para que la FSM complete este recorrido con un único byte, el programa debe ejecutar el bloque loop() consecutivamente a través de las siguientes etapas:

- **Ciclo 1 (IDLE → WRITE):** Se detecta el dato, se guarda la marca tiempo_recepcion = micros() y se transiciona a WRITE. Al salir del switch se ejecuta el primer T_{delay}
- **Ciclo 2 (WRITE → IDLE):** Se ejecuta la escritura en la RAM y se regresa a IDLE. Se ejecuta el segundo T_{delay} .

- **Ciclo 3 (IDLE → READ):** Al estar los contadores desiguales, la FSM transiciona a READ. Se ejecuta el tercer T_{delay} .
- **Ciclo 4 (READ → ECHO):** Se extrae el dato de la memoria hacia la variable temporal y se pasa a ECHO. Se ejecuta el cuarto T_{delay} .
- **Ciclo 5 (ECHO):** Al iniciar el estado se calcula la diferencia micros() - tiempo_recepcion.

Matemáticamente, el tiempo retenido acumula exactamente **4 periodos de retardo:**

$$Latencia_{mín} = 4 \times T_{delay} = 4 \times 10 \text{ ms} = 40\text{ms} = 40\,000 \text{ us}$$

- **Latencia Máxima Teórica (Ráfaga de N bytes en memoria)** Es el tiempo de espera en el peor escenario para el último byte alojado en una ráfaga. Cada byte previo almacenado en el buffer requiere de 3 ciclos de loop (IDLE → READ → ECHO) para ser desalojado, sumando 30 ms de retraso por cada uno. Para un número n de bytes acumulados antes del byte evaluado, la fórmula es

$$Latencia_{máx} = (n \times 3 \times T_{delay}) + Latencia_{mín}$$

Para el 2do byte de una ráfaga (n = 1 byte previo en espera):

$$Latencia_{máx} = (1 \times 3 \times 10 \text{ ms}) + 40\text{ms} = 70\text{ms} = 70\,000 \text{ us}$$

- **Tiempo crítico de desbordamiento por inundación ($T_{overflow}$)** :Ventana de tiempo máxima en la que una ráfaga continua externa satura las 15 posiciones útiles del buffer antes de activar la alerta del Pin 13:

$$t_{overflow} = 15 \times T_{byte} = 15 \times 1.0416\text{ms} = 15.624 \text{ ms}$$